

Trabalho Prático: TAD, Recursividade e Gestão de Memória

1 Contexto do Projeto

A startup de tecnologia agrícola **AgroTech** está desenvolvendo um novo sistema de mapeamento para drones autônomos (o *AgroDrone*). Para garantir a eficiência do processamento embarcado, a equipe de engenharia determinou que não poderão ser utilizadas estruturas de dados lineares complexas (como listas ou pilhas) nesta etapa. O sistema base foca apenas no encapsulamento estrito dos dados, chamadas recursivas para cálculo de área e controle sobre referências de memória.

O sistema precisa processar e avaliar "Lotes" de terra que, para fins de análise, podem sofrer subdivisões.

2 Requisitos Técnicos (Questão Única)

Implementar o sistema base na linguagem Java, aplicando rigorosamente os conceitos a seguir.

Parte A: TAD e Alocação de Memória (2,0 pts)

- Defina um Tipo Abstrato de Dados (TAD) chamado **Lote**. Este TAD deve conter: um ID (inteiro), a **área_base** (ponto flutuante), e um *array* fixo para armazenar a leitura de 4 sensores climáticos.
- Demonstre, através de comentários explicativos no código, a diferença entre a **alocação estática** (espaço de tamanho fixo reservado em tempo de compilação/stack, como os atributos primitivos e o array estático) e a **alocação dinâmica** (espaço requisitado no heap em tempo de execução para instanciar o objeto **Lote**).

Parte B: Referências e Alias (2,0 pts)

- Crie um método ou função **analisarImpacto(Lote a, Lote b)** que demonstre intencionalmente o conceito de **Alias** (quando duas ou mais referências distintas apontam para o mesmo endereço de memória).
- Dentro da rotina, modifique um atributo utilizando uma das referências e exiba o valor utilizando a outra, provando que ambas compartilham a mesma alocação.
- Adicione um comentário justificando o impacto prático (e o possível risco de efeitos colaterais) desse comportamento no software.

Parte C: Recursividade (3,0 pts)

- Implemente uma função **estritamente recursiva** chamada **calcularAreaUtil(int nivel, double area)**.

- A regra de negócio define que: no **nível 0** (caso base), a função apenas retorna a própria **area**. Para níveis maiores, o lote é dividido em 4 sub-lotes, mas o drone identifica uma perda de 5% de área útil a cada nível de subdivisão (devido a corredores de segurança).
- A função deve chamar a si mesma, reduzindo o nível, até calcular e retornar a soma da área útil no nível de profundidade requisitado. **É terminantemente proibido o uso de laços de repetição (for, while).**

3 Critérios de Avaliação

A pontuação total de 7,0 pontos será distribuída conforme a tabela abaixo:

Critério	Pontuação
Definição do TAD Lote e distinção correta entre alocação estática e dinâmica	2,0
Demonstração e explicação correta do conceito de Referências/Alias	2,0
Lógica correta e implementação estritamente recursiva da função de área	3,0
Total	7,0

4 Instruções de Entrega

- O código deve ser organizado e conter uma rotina **Main** que instancie os elementos e demonstre, no console, o funcionamento das Partes A, B e C.
- É obrigatório o uso de comentários para as justificativas teóricas solicitadas.
- Formato: Arquivo compactado (.zip) contendo os arquivos fonte.
- Prazo: Até o final da aula.